People's Democratic Republic of Algeria

University of Sciences and Technology Houari Boumediene

Computer Science Faculty



Introduction to Algorithms Analysis (Complexity)

Data Structures and Algorithms (L2 - Computer Science)

Edited by: Dr. B. BESSAA

Definition

An algorithm is a **finite** and **unambiguous** sequence of operations or instructions to solve **a class** of problems.

Solving quadratic equation	Instance of the problem
$a\,X^2 + b\,X + \;\; c = 0$	$a = 2, b = 5, c = -7$

When designing and then studying an algorithm, **two notions** are extremely important :

1- **Correctness** of the algorithm: does it do what we want? Does it end?

2- **Efficiency** of the algorithm: how fast does it run? Is it optimal?

Computing the **complexity** (temporal or/and spatial) is used to compare the performance of different **correct** algorithms solving the **same problem**.

Consider the following two algorithms to permute two integers.

```
Algorithm Permute1;
Var  A,B,C:integer;
Begin
        Read(A,B);
        C←A; A←B; B←C;
        Write(A,B);
End.
```

Space : **3** Integer Variables.
Time : **3** Assignments.

```
Algorithm Permute2;
Var  A,B:integer;
Begin
        Read(A,B);
        A←A+B; B←A-B; A←A-B;
        Write(A,B);
End.
```

Space : **2** Integer Variables.
Time : **3** Assignments.
3 Operations.

Currently, we consider almost **exclusively** the **time complexity**. Except for situations where memory space is **restricted** and/or **expensive**.

In this course we will focus on the Time Complexity.

Factors influencing the running time

Hardware Architecture

Computer Type	Time (ms)
Personal computer	52
Server	11,5
Supercomputer	0,085

The language and its compiler

Speed of compilers according to a study in 2017

C < Rust < C++ < Ada < Java < Chapel < Go < Pascal …

Algorithm structure

Loops, Nested Loops, …

Data structures used

Arrays, Lists, Files, …

The type and size of data used

Number of values, specific values, Sorting, …

And so, on what factors will we base the evaluation?

Actually, the evaluation should be done **independently** of the environment.

To be fair, complexity should be measured based on the **number of elementary operations** and the **size of input data**.

What are elementary Operations?

Well, **elementary operations** are **basic** operations: Assignment, comparison (=, >,<, …), arithmetic operations (+,-,x,/), Input/Output (Read and Write).

How to measure the running time of an algorithm (Time complexity) ?



Approach 1 :

Experimental studies

The experimental study consists of:

- Writing a program **implementing** the algorithm.

- Executing the program with data sets of **various** **sizes** and **content.**

- Calculating the runtime for **each case.**

Limitation of the experimental approach

- Need to **Implement** the algorithm(s).

- The set of input data is **limited** (necessarily **finite**).

- The set of input data (instances) **does not cover** all possible cases.

Finally, the result **depends** on the experimental **environment**, that is why calculating the exact complexity may prove to be **unnecessary**.

Theoretical Analysis

The Theoretical Analysis consists of :

Define a **general methodology** for analyzing the running time of the algorithm which :

- Uses the algorithm description (**no need to implement**).
- Characterizes the running time as a **function** of the input data **size** (**T(n)**).
- Consider the **different** possible cases of input data.

Brief

We no longer study the **obtained results** but the **way** in which these results **evolve** according to the **size** of the input.

Example

Find the **index** of a value **V** in an array **A** of ***n*** integers.
The part of the algorithm that searches for the index **I** is:

```
I←1;
While I ≤ n And A[I]≠V Do I← I + 1; Endwhile;
```

Calculating the theoretical complexity involves determining the number of basic operations to be executed with respect to the input size (**n**).

Consider the following instance: **A =**

7	8	20	10	45

, n=5

For **V = 7**, we execute **one** assignment (I←1) and **2** tests (1≤5, A[1]≠7), which gives **3 actions**.

For **V = 3**, we execute **one** assignment (I←1) and we execute the loop **5 times**. At each time, we have **2** tests, **1** assignment and **1** addition (**4** operations) and **1** test at the end, so a total of 2+5x4 = **22 actions**.

What value will we take ?

This example leads us to define the different types of complexity.

Indeed, there is often an (**infinite**) number of **possible instances**, and depending on the considered instance , an algorithm **can take more** or **less** time.

The possible instances can then be **classified** into **three** cases:

1- Best-case.

2- Worst-case.

3- Average-case.

In our example :

The case **V=7** was the **Best-case (the value is at the front).**

The case **V=3** was the **Worst-case (the value does not exist).**

In complexity analysis, we **generally focus** on the **worst-case** time complexity as it does not **require any specific conditions** on the input data.

In fact, we aim to determine an **upper bound** on the running time.

Going back to the previous example, in the **worst-case** for an array of size **n**, we execute the **4** loop **operations n** times, in addition to the assignment **before** the loop and the **final test**. This gives us the number of operations as a function of **n** defined by the function:

$$T(n) = 4n + 2$$

We call the **function T**: The **Time complexity function**.

Example

```
Algorithm A1;
Var  nb, i, n : integer ; T1, T2: Array[1000] of integer;
Begin
      Read (n); nb←0 ;                                   2  (Read + assignment)
      For i←1 To n Do Read(T1[i]); Endfor ;
      For i←1 To n Do Read(T2[i]); Endfor ;
      For i←1 To n
      Do If (T1[i]=T2[i]) Then nb ←nb+1 ; EndIf ;
      Endfor ;
      Write (nb) ;
End.
```

Note

```
For  i←1 To n
Do
        <Action>;
Endfor;
```

```
i←1 ;                    1  assignment
While  i ≤ n             n+1 tests
Do
        <Action>;
        i← i +1 ;        2n (assignment+addition)
Endwhile;
```

For "**For" loops**, we implicitly have an **increment** of the counter (addition + assignment) and a **test** at each iteration, in addition to a **test at the end** to exit the loop, and an **assignment** at the beginning to initialize the counter. Therefore, we have **3n+2** operations.

If we consider that the incrementation of the counter **i** as a basic action, the total number will be **2n+2**

Going back to the previous example, in the **worst-case** for an array of size **n**, we execute the **4** loop **operations n** times, in addition to the assignment **before** the loop and the **final test**. This gives us the number of operations as a function of **n** defined by the function:

$$T(n) = 4n + 2$$

We call the **function T**: The **Time complexity function**.

Example

```
Algorithm A1;
Var  nb, i, n : integer ; T1, T2: Array[1000] of integer;
Begin
      Read (n); nb←0 ;                                   2  (Read + assignment)
      For i←1 To n Do Read(T1[i]); Endfor ;              3n+2 (For loop) + n  (Read)
      For i←1 To n Do Read(T2[i]); Endfor ;              3n+2 (For loop) + n  (Read)
      For i←1 To n                                       3n+2 (For loop)
      Do If (T1[i]=T2[i]) Then nb ←nb+1 ; EndIf ;        n  (test) + 2n (assignment+addition)
      Endfor ;
      Write (nb) ;                                       1 (Write)
End.
```

Total : **T(n) = 14 n + 9**

Example

```
Algorithm SumFac;
Var  S,F, i, n : integer ;
Begin
      Read (n); F ←1 ; S ←0 ;
      For i←1 To n
      Do     F ← F * i ;
             S ← S + F ;
      Endfor ;
      Write (S) ;
End.
```

3 (read + 2 assignment)

3n+2 (For loop)

2n (assignment+multiplication)

2n (assignment+addition)

1 (write)

Total : **T(n) = 7 n + 6**

Does the function T really give us the running time of the algorithm?

In reality, the function **T** gives us the number of elementary operations to be performed. To find the time, we need to multiply it by a certain constant **C** which represents the execution time of a single elementary operation.

The constant **C** **depends** on the environment (Hardware, Compiler, etc.)

Therefore, different values of the constant **C** yield different values of the running time. For example, for : **T(n)=4n+2**

With **C1**, the time is : **(4C1)n + 2C1**
With **C2**, the time is : **(4C2)n + 2C2**
With **C3**, the time is : **(4C3)n + 2C3**

} **of the form: a.n + b**

Where **a** and **b** are constants that do not depend on the problem size **n**.

When studying the **complexity** of an algorithm, we focus on **large sizes** that tend towards **infinity**. In this case, the constants become negligible.

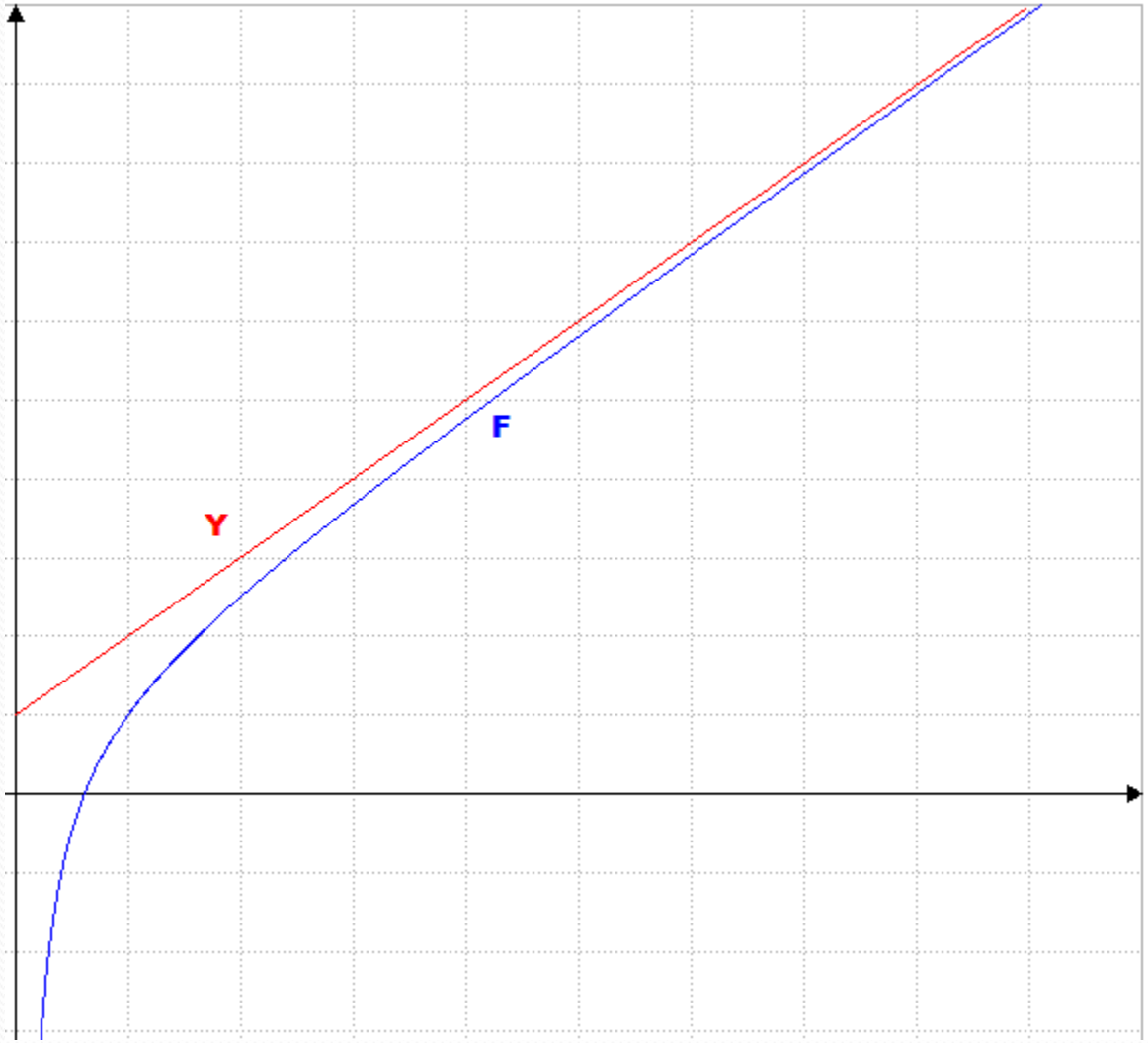
Thus, a function of the form **T(n) = a n + b** will have the same order of magnitude as a function of the form **T(n) = n**.

We are talking about a behavior

ASYMPTOTIC

$F(x) = x+1 - 1/x$ $\quad Y = x+1$

$\lim_{x \to \infty} (F(x) - Y) = 0$, hence Y is an asymptote



When studying the **complexity** of an algorithm, we focus on **large sizes** that tend towards **infinity**. In this case, the constants become negligible.

Thus, a function of the form **T(n) = a n + b** will have the same order of magnitude as a function of the form **T(n) = n**.

We are talking about a behavior

ASYMPTOTIC

Asymptotic Complexity?

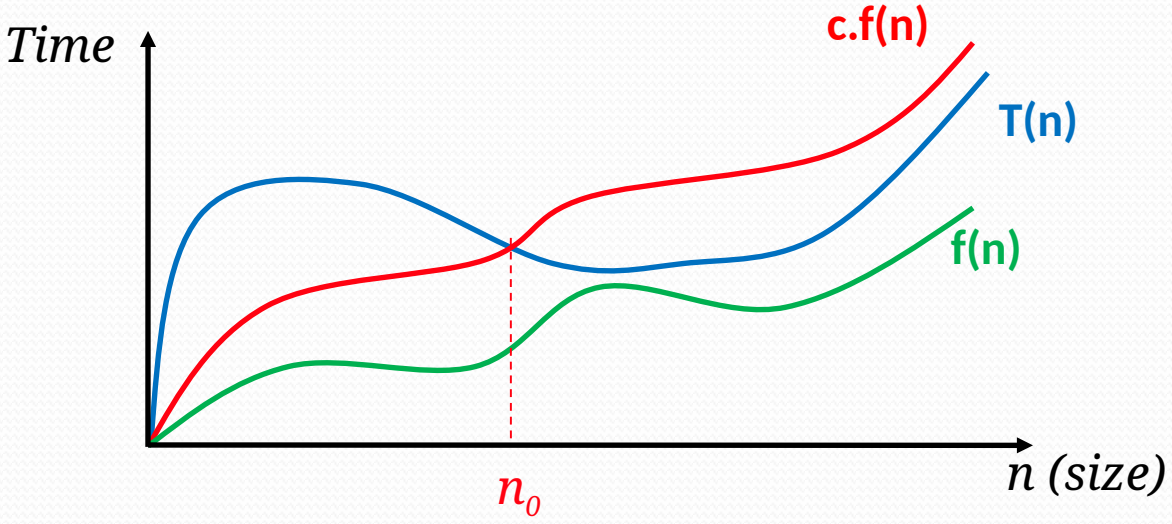
Asymptotic complexity is an **approximation** of the number of operations performed by an algorithm for a given input dataset. It is called '**asymptotic**' because it **considers** large data and **retains** only the **most significant** term in the formula, **discarding** lower-order terms and constant multipliers.

Example: **$T(n) = 4n^2+5n+3$** will have the **same** growth rate as : **n^2**

How to measure asymptotic complexity ?

The **asymptotic complexity** of an algorithm involves determining the **smallest** upper bound that **bounds** the time complexity function of the algorithm.

Let **T(n)** be the time function, the goal is to find a function **f(n)** that serves as an **upper bound** of **T(n)** up to a multiplicative constant **c** starting from a certain value n_0 of **n**.



Landau Notation

Landau notation are a family of mathematical notations used to compare asymptotic functions. There is several notations, Big-Oh (**O**), Big-Omega (Ω), Big-Theta (θ), Little-o (o),...

In Computer science, big **O** notation is used to classify algorithms according to how their **running time** grows as the **input size** grows.

Let **T(n)** be the time function as a function of the problem size **n**. Definning the asymptotic complexity of **T** is equivalent to finding a **simple function f** that upper-bounds the function **T**. In this case, we say that the complexity of **T** is **O**(f(n)) (**Big-Oh of f(n)**).

Formal definition

$T(n) \in O(f(n))$ iff $\exists\, c > 0, \exists\, n_0 \geq 0$ such that $\forall n \geq n_0$, $T(n) \leq c.f(n)$

Example : Let $T(n) = 15n^2 + 10$

We have for each n $15n^2 \leq 15n^2$

And for $n \geq 4$ $10 \leq n^2$

Taking the sum of the two sides, we have, for $n \geq 4$, $T(n) \leq 16n^2$

Let $c = 16$, $n_0 = 4$ and $f(n) = n^2$, so **$T(n) = O(n^2)$**

Time functions classification

There are commonly used time complexity functions, and they are classified as follows:

1- **Constant** : T(n) = **O**(1)

2- **Logarithmic** : T(n) = **O**(Log(n))

3- **Polynomial** : T(n) = **O**(n^K)

K=1 : **Linear** T(n) = **O**(n)

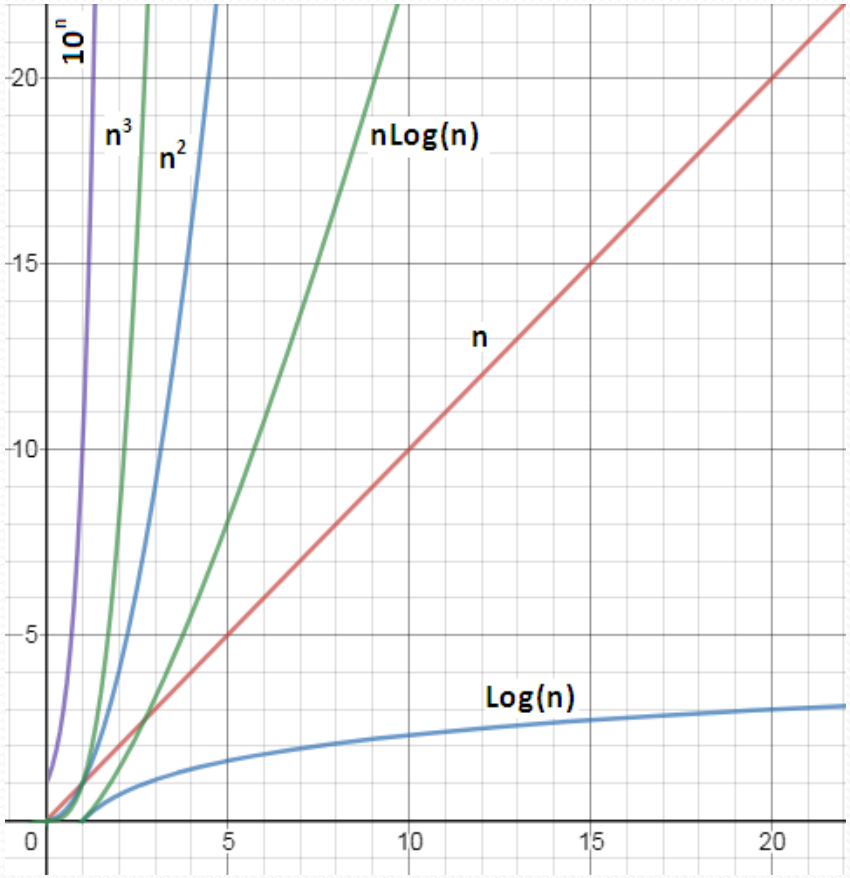
K=2 : **Quadratic** T(n) = **O**(n^2)

K=3 : **Cubic** T(n) = **O**(n^3)

4- **Linearithmic** T(n) = **O**(nLog(n))

5- **Exponential** T(n) = **O**(a^n).

O(Log(n)) < O(n) < O(nLog(n)) < O(n^2) < O(n^3) < O(a^n)



Typical growth rates

Properties

- Let k a nonzero constant, then $O(k) = O(1)$

  $O(kF) = O(F)$.

- Sum : $O(F1) + O(F2) = Max(O(F1), O(F2))$.
- Product : $O(F1) * O(F2) = O(F1*F2)$.
- If statement : **If** <Cond> **Then** F1 **Else** F2 **Endif** ; : $Max(O(F1), O(F2))$.
- Loops : **For** I←1 **To** n **Do** F ; **Endfor** ; : $O(nF)$.

Example

```
Algorithm SumFac;
Var  S,F, i, n : integer ;
Begin
      Read (n); F ←1 ; S ←0 ;
      For i←1 To n
      Do    F ← F * i ;
            S ← S + F ;
      Endfor ;
      Write (S) ;
End.
```

O(3) = O(1)

O(4n) = O(n)

O(1)

Finally : the algorithm's complexity is : $O(1)+ O(n)+ O(1) = O(\mathbf{n})$

Practical order of magnitude

For a computer that performs **10^6** operations per second, the estimated running time for a problem size **n** is given in the following table for different time complexity functions:

Size	$Log_2(n)$	n	$n\ Log_2(n)$	n^2	2^n
10	0,003 ms	0,01 ms	0,03 ms	0,1 ms	1 ms
10^2	0,006 ms	0,1 ms	0,6 ms	10 ms	10^{14} centuries
10^3	0,01 ms	1 ms	10 ms	1 s	
10^4	0,013 ms	10 ms	0.1 s	100 s	
10^5	0,016 ms	100 ms	1.6 s	3 hours	
10^6	0,02 ms	1 s	20 s	10 days	